Informatika és távközlés ágazat

Stringify

Készítette
Kósa Áron
Ricz Attila
Szalonna József
Szoftverfejlesztő és -tesztelő szak

Témavezető
Kerényi Róbert Nándor
oktató

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés **4**

1.1. A projekt bemutatása és célja 4

1.2. Célközönség meghatározása 5

1.3. Alkalmazott technológiák és eszközök 6

2. Követelményanalízis **7**

2.1. Funkcionális követelmények 7

2.2. Nem funkcionális követelmények 7

2.3. Használati esetek (Use Cases) 8

3. Rendszertervezés **9**

3.1. Adatbázis terv 9

3.1.1. ER-diagram és kapcsolatok 9

3.1.2. Adatszótár és táblaleírások 10

3.2. Rendszerarchitektúra és komponensek 11

3.2.1. ORM (Object-Relational Mapping) 11

3.2.2. Entity Framework 11

3.3. REST API végpontok tervezése 11

3.4. UI/UX tervek (Wireframes) 12

4. Megvalósítás **13**

4.1. Adatbázis réteg implementációja 13

4.2. REST Backend fejlesztése 14

4.2.1. Adatkezelés és Üzleti logika 15

4.2.2. Biztonság és Autentikáció (JWT) 15

4.3. React Frontend fejlesztése 15

4.3.1. Komponens-architektúra 16

4.3.2. Reszponzív megoldások és Mobil nézet 16

4.4. WPF Desktop alkalmazás 17

4.4.1. MVVM architektúra és Adatkötés 17

4.4.2. Törzsadat-karbantartó modulok 17

5. Minőségbiztosítás és Tesztelés **18**
5.1. Unit tesztek (Egységtesztelés) 18
5.2. API tesztelés (Postman/Swagger) 19
5.3. Felhasználói felület és Reszponzivitás tesztelése 19
5.4. Hibajegyzék és javítások 20

6. Felhasználói útmutató **21**
6.1. Rendszerkövetelmények és Telepítés 21
6.2. A szoftver használata (Web/WPF) 23

7. Összegzés **25**
7.1. Projekt értékelése 25
7.2. Továbbfejlesztési lehetőségek 26

8. Mellékletek **29**
8.1. Backend unit tesztek 29

1. fejezet

Bevezetés

1.1. A projekt bemutatása és célja

A *Stringify* projekt célja egy olyan webalapú alkalmazás megvalósítása, amely egyedi elektromos gitárok konfigurálását és online megrendelését teszi lehetővé. Az alkalmazás központi eleme egy úgynevezett *guitar builder*, amely segítségével a felhasználók különböző testformák, faanyagok, hardverelemek, elektronikai komponensek és megjelenési opciók közül választhatnak.

A rendszer nem csupán konfigurátorként működik, hanem egy webshop funkcióval is kiegészül, amely lehetővé teszi az elkészített egyedi gitárok kosárba helyezését, megrendelését és nyomon követését. A kiválasztott alkatrészek és opciók alapján az alkalmazás automatikusan kiszámítja a végleges árat, ezzel támogatva a felhasználók döntéshozatalát.

A projekt célja, hogy a hangszertervezés és -vásárlás folyamatát digitális környezetbe helyezze, és egy vizuálisan szemléletes, felhasználóbarát felületen keresztül tegye elérhetővé mind a konfigurálást, mind a vásárlást. A rendszer moduláris felépítése lehetővé teszi a későbbi bővíthetőséget, például új terméktípusok, kiegészítők vagy fizetési megoldások integrálását.

A *Stringify* tehát egy komplex webes megoldás, amely összekapcsolja a felhasználói élményt, az üzleti logikát és az adatkezelést egy egységes platformon.

1.1. ábra. A Stringify webalkalmazás főoldala

1.2. Célközönség meghatározása

A *Stringify* webalkalmazás elsődleges célközönségét olyan gitározással foglalkozó felhasználók alkotják, akik egyedi megjelenésű és konfigurációjú elektromos gitárt szeretnének tervezni és megrendelni online felületen keresztül.

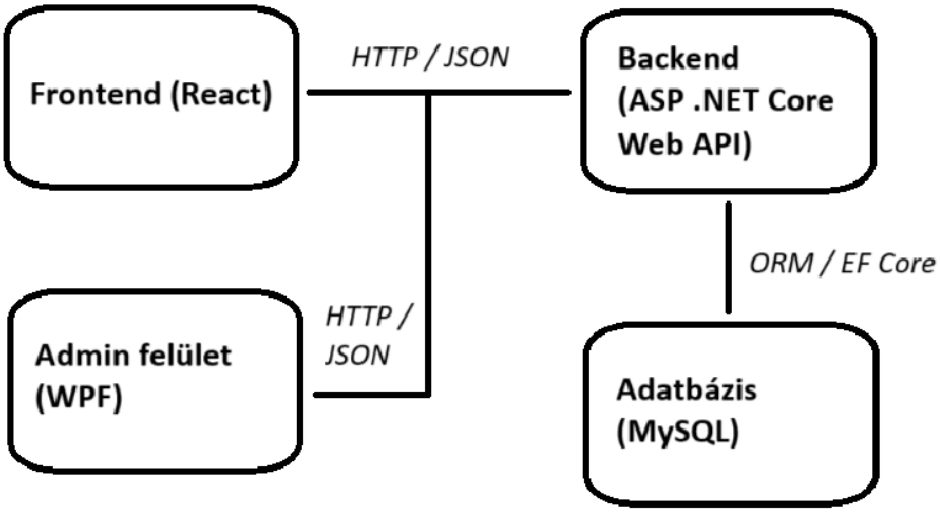
A rendszer elsősorban hobbi- és félprofi gitárosokat céloz meg, akik nyitottak az egyedi hangszerösszeállításokra, de nem rendelkeznek szakmai hangszerkészítői ismeretekkel. Számukra kiemelten fontos, hogy az alkalmazás vizuálisan szemléletes módon mutassa be a különböző választható opciókat, valamint egyértelmű információt nyújtson az árakról és a rendelési folyamatról.

Másodlagos célközönségként megjelennek a kisebb hangszerboltok és egyedi gitárkészítők, akik a rendszert online értékesítési és megrendeléskezelési felületként használhatják. Számukra a webshop funkciók, az adminisztrációs felület, valamint a termék- és árkezelés jelent hozzáadott értéket.

A felhasználói felület kialakítása során kiemelt szempont volt az áttekinthetőség, az egyszerű használhatóság, valamint az, hogy a technikai háttérrel nem rendelkező felhasználók számára is könnyen értelmezhető legyen a gitártervezési és vásárlási folyamat.

1.3. Alkalmazott technológiák és eszközök

- **Adatbázis:** MySQL
- **Backend:** RESTful API (pl. ASP.NET Core / Node.js)
- **Webes kliens:** React (Reszponzív kialakítás)
- **Asztali kliens:** WPF (C#)



1.2. ábra. Rendszer architektúra

2. fejezet

Követelményanalízis

A követelményanalízis célja a fejlesztendő rendszerrel szemben támasztott elvárások részletes meghatározása. Ebben a fejezetben bemutatásra kerülnek a rendszer funkcionális és nem funkcionális követelményei, valamint a felhasználók és a rendszer közötti interakciókat leíró használati esetek.

2.1. Funkcionális követelmények

A *Stringify* webalkalmazás funkcionális követelményei azoknak a konkrét szolgáltatásoknak és műveleteknek az összességét jelentik, amelyeket a rendszernek biztosítania kell a felhasználók számára.

A rendszernek az alábbi fő funkciókat kell támogatnia:

- Felhasználói regisztráció és bejelentkezés.
- Egyedi elektromos gitár konfigurálása előre definiált opciók (testforma, faanyag, pickupok, színek stb.) kiválasztásával.
- A gitár vizuális megjelenítésének frissítése a kiválasztott konfiguráció alapján.
- A konfiguráció összárának automatikus kiszámítása valós időben.
- Az elkészített gitár mentése és kosárba helyezése.
- Megrendelések kezelése és visszajelzés a felhasználó számára.
- Adminisztrátori felület biztosítása a törzsadatok és árak karbantartásához.

2.2. Nem funkcionális követelmények

A nem funkcionális követelmények a rendszer működésének minőségére, használhatóságára és technikai jellemzőire vonatkoznak.

A rendszerrel szemben támasztott legfontosabb nem funkcionális követelmények a következők:

- **Használhatóság:** Az alkalmazás kezelőfelületének áttekinthetőnek és könnyen használhatónak kell lennie technikai háttérrel nem rendelkező felhasználók számára is.
- **Teljesítmény:** A konfiguráció módosítására adott válaszidő ne haladja meg az 1 másodpercet.
- **Skálázhatóság:** A rendszernek képesnek kell lennie több felhasználó egyidejű kiszolgálására.
- **Biztonság:** A felhasználói adatok védelmét hitelesítési és jogosultságkezelési mechanizmusokkal kell biztosítani.
- **Platformfüggetlenség:** A webes kliensnek különböző böngészőkben és eszközökön is megfelelően kell működnie.

2.3. Használati esetek (Use Cases)

A használati esetek bemutatják, hogy a felhasználók milyen módon lépnek kapcsolatba a rendszerrel, valamint hogy a rendszer hogyan reagál ezekre az interakciókra.

A rendszer legfontosabb használati esetei az alábbiak:

- Felhasználó regisztrációja és bejelentkezése.
- Egyedi gitár tervezése és konfiguráció módosítása.
- Ár kalkuláció megtekintése.
- Megrendelés leadása.
- Adminisztrátor általi adatmódosítás és karbantartás.

A használati esetek részletesebb bemutatása és vizuális ábrázolása (use case diagram) a rendszertervezési fejezetben kerül bemutatásra.

3. fejezet

Rendszertervezés

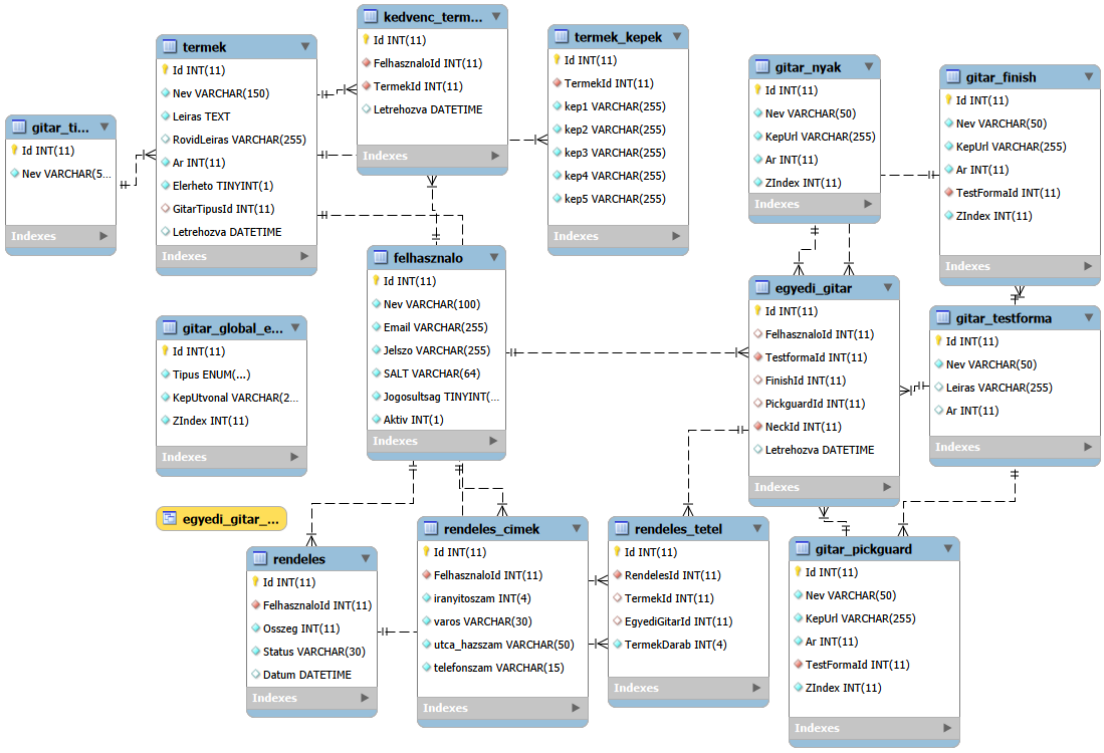
Ebben a fejezetben a rendszer adatbázis- és architekturális tervezése kerül bemutatásra. A cél egy olyan webes alkalmazás megvalósítása, amely egyszerre lát el hagyományos webshop funkciókat, valamint egy egyedi elektromos gitár konfigurátor működtetését.

3.1. Adatbázis terv

Az adatbázis relációs modellben került megtervezésre MySQL/MariaDB adatbázis-kezelő rendszer használatával. A tervezés során fontos szempont volt az adatok logikus elkülönítése, a redundancia minimalizálása, valamint a rendszer hosszú távú bővíthetőségének biztosítása.

3.1.1. ER-diagram és kapcsolatok

Az adatbázis logikai felépítését az 3.1 ábrán látható ER-diagram szemlélteti. A diagram a rendszer fő entitásait és azok kapcsolatait mutatja be, különválasztva a webshop modul és az egyedi gitár konfigurátor adatstruktúráját.



3.1. ábra. Az adatbázis logikai ER-diagramja

A webshop modul központi elemei a termékek, a felhasználók és a rendelések, míg az egyedi gitár konfigurátor modul az egyedi gitár összeállításához szükséges vizuális komponenseket tartalmazza. Az egyedi gitár entitás idegen kulcsokon keresztül kapcsolódik a választható gitártest formákhoz, testfinishekhez, pickguardokhoz és nyakakhoz.

A diagram logikai ER-diagramként szolgál, ezért nem tartalmaz minden adatmezőt, csupán a legfontosabb entitásokat és relációkat jeleníti meg az áttekinthetőség érdekében.

3.1.2. Adatszótár és táblaleírások

Az adatbázis központi táblája az `egyedi_gitar`, amely a felhasználó által összeállított gitár konfigurációt írja le.

egyedi_gitar

- `Id` – elsődleges kulcs
- `FelhasznaloId` – a konfigurációt létrehozó felhasználó azonosítója
- `TestformaId` – a gitár formai típusa (pl. Stratocaster, Telecaster)
- `FinishId` – a kiválasztott teljes gitártest képe
- `PickguardId` – az előre huzalozott pickguard képe

– `NeckId` – a gitár nyakának képe

– `Letrehozva` – a konfiguráció létrehozásának időpontja

A `gitar_testforma` tábla kizárólag a gitár formai kategóriáját határozza meg, képi adatokat nem tartalmaz. A `gitar_finish` és `gitar_pickguard` táblák a formafüggő, előre kivágott képi elemeket tárolják, míg a `gitar_nyak` tábla olyan nyakakat tartalmaz, amelyek bármely testformával kompatibilisek.

3.2. Rendszerarchitektúra és komponensek

Az alkalmazás háromrétegű architektúrát követ, amely elkülöníti a felhasználói felület, az üzleti logika és az adatelérés feladatait.

– Prezentációs réteg (frontend)

– Alkalmazási réteg (backend)

– Adatelérési réteg (adatbázis)

3.2.1. ORM (Object-Relational Mapping)

Az ORM (Object-Relational Mapping) célja az objektumorientált programozási modell és a relációs adatbázis közötti különbségek áthidalása. Az ORM lehetővé teszi, hogy az adatbázis táblák objektumokként jelenjenek meg a programkódban, csökkentve az adatbázis-kezeléssel kapcsolatos hibalehetőségeket.

3.2.2. Entity Framework

A backend megvalósítása során az Entity Framework ORM keretrendszer kerül alkalmazásra. Az adatbázis táblák entitásokként jelennek meg, az idegen kulcs alapú kapcsolatok pedig navigációs tulajdonságokon keresztül érhetők el.

Az Entity Framework támogatja az adatbázis-migrációkat, valamint biztosítja az adatok konzisztens kezelését a teljes alkalmazáson belül.

3.3. REST API végpontok tervezése

A rendszer REST architektúra szerint működő API-t biztosít, amely JSON formátumban kommunikál a frontend alkalmazással. Az API végpontok lehetővé teszik a termékek, a konfigurálható gitár elemek lekérdezését, valamint az egyedi gitár konfigurációk mentését és visszatöltését.

3.4. UI/UX tervek (Wireframes)

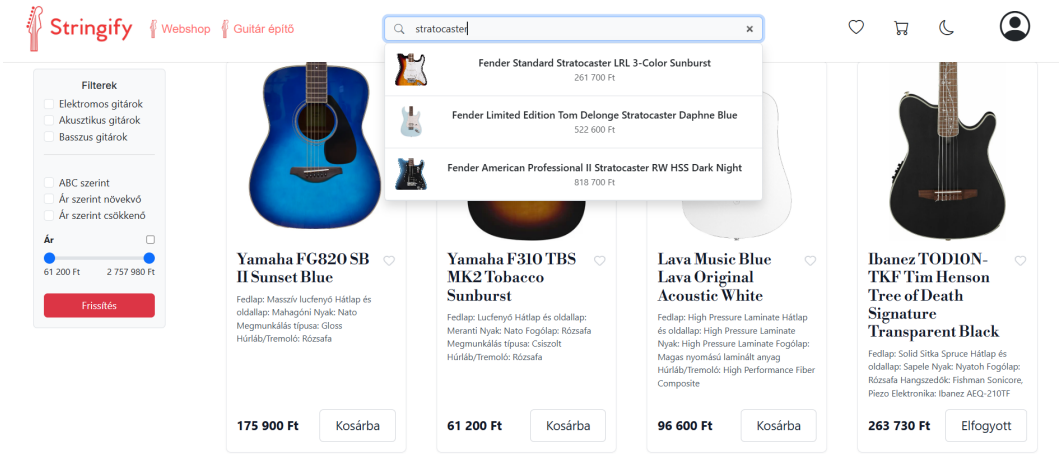
A felhasználói felület tervezése során kiemelt szempont volt az átláthatóság és az egyszerű kezelhetőség. Az egyedi gitár konfigurátor réteges megjelenítést alkalmaz, ahol a gitár egyes elemei egymásra helyezve jelennek meg.

A wireframe tervek meghatározzák az oldalak szerkezetét, a fő funkcionális elemek elhelyezkedését, valamint a felhasználói navigáció alapelveit.

4. fejezet

Megvalósítás

Ebben a fejezetben a rendszer konkrét implementációja kerül bemutatásra. A projekt három fő komponensből áll: egy REST alapú backend alkalmazásból, egy React alapú webes frontendből, valamint egy WPF technológiával készült desktop adminisztrációs alkalmazásból.



4.1. ábra. Gitár webshop terméklista oldal szűrőkkel, keresővel

4.1. Adatbázis réteg implementációja

Az adatbázis MySQL/MariaDB alapokon került megvalósításra. A relációs modell a tervezési fázisban kialakított ER-diagram alapján készült el.

Az adatbázis törzsadatai külön táblákban kerülnek tárolásra, mint például:

- `gitar_testforma`
- `gitar_finish`
- `gitar_pickguard`

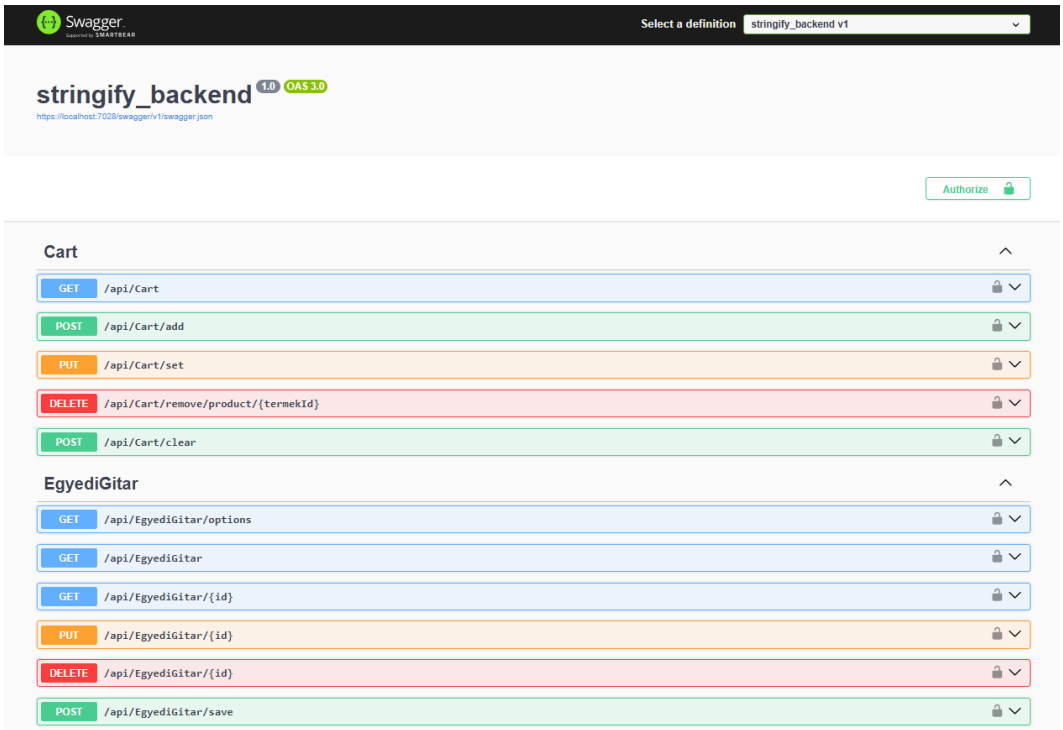
– `gitar_nyak`

Az egyedi konfigurációkat az `egyedi_gitar` tábla tárolja, amely idegen kulcsokon keresztül kapcsolódik a kiválasztott komponensekhez.

A grafikai elemekhez tartozó képfájlok fizikailag a webszerveren kerülnek tárolásra, míg az adatbázis kizárólag azok URL elérési útját rögzíti. Ez lehetővé teszi a képek rugalmas cseréjét az adatbázis módosítása nélkül.

Az egyedi gitár árkalkulációja nézet (VIEW) segítségével történik, amely a fizikai alkatrészek árait összegzi. A vizuális elemek (finish, pickguard) nem vesznek részt az árképzésben.

4.2. REST Backend fejlesztése



4.2. ábra. REST API végpont részletek Swagger dokumentációban

A backend ASP.NET alapokon került megvalósításra, REST architektúrát követve. A kliens és a szerver JSON formátumban kommunikál.

A rendszer főbb végpontjai:

– `/api/termekek`

– `/api/egyedigitar`

– `/api/testforma`

- `/api/finish`
- `/api/pickguard`
- `/api/nyak`

A backend réteg felelős az adatok validálásáért, a konfigurációk mentéséért, valamint a rendelési folyamat kiszolgálásáért.

4.2.1. Adatkezelés és Üzleti logika

Az adatelérés Entity Framework ORM használatával történik. Az adatbázis táblák entitásosztályokként jelennek meg az alkalmazásban.

Az üzleti logika réteg felelős:

- az egyedi gitár konfiguráció validálásáért
- az árkalkuláció végrehajtásáért
- a rendelési állapotok kezeléséért

A konfigurátor működése során a kiválasztott elemek azonosítói kerülnek mentésre, nem pedig a képfájlok vagy grafikai adatok, ezáltal biztosítva az adatbázis normalizált szerkezetét.

4.2.2. Biztonság és Autentikáció (JWT)

A felhasználói autentikáció JSON Web Token (JWT) alapokon történik. Sikeres bejelentkezés esetén a szerver aláírt tokent generál, amely a kliensoldalon kerül tárolásra.

A védett végpontok csak érvényes token birtokában érhetők el, biztosítva a jogosultság alapú hozzáférés-vezérlést. A token tartalmazza a felhasználó azonosítóját és szerepkörét, amely alapján a backend eldönti, hogy a felhasználó hozzáférhet-e az adott erőforráshoz.

4.3. React Frontend fejlesztése

A webes kliens React keretrendszerrel készült, komponens-alapú felépítéssel.

A gitár konfigurátor réteges megjelenítést alkalmaz. A kiválasztott elemek képei egymásra helyezve jelennek meg réteges (layer alapú) megjelenítéssel., CSS alapú pozicionálással és z-index használatával.

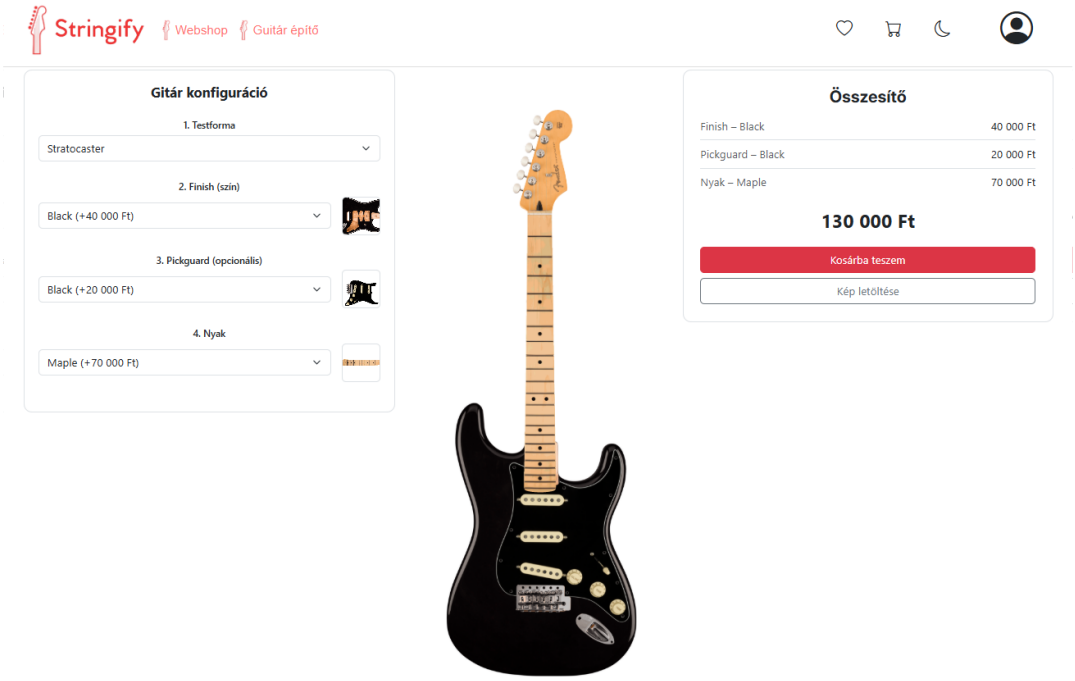
A megjelenítési sorrend:

- Test (finish)

– Pickguard

– Nyak

A frontend kizárólag az adott testformához tartozó opciókat jeleníti meg, biztosítva a kompatibilitást.



4.3. ábra. Egyedi gitár konfigurátor felülete

4.3.1. Komponens-architektúra

A frontend főbb komponensei:

– Konfigurátor oldal

– Terméklista oldal

– Kosár

– Bejelentkezés / Regisztráció

A konfigurátor belső komponensekre bontva valósul meg, például külön kezelve a test kiválasztását, a pickguard kiválasztását és a nyak kiválasztását.

4.3.2. Reszponzív megoldások és Mobil nézet

A frontend felület kialakítása során a Bootstrap keretrendszer került alkalmazásra, amely beépített reszponzív grid rendszert és előre definiált komponenseket biztosít.

A layout kialakítása elsősorban Bootstrap osztályok (`container`, `row`, `col-*`) használatával történt, így az oldal külön CSS szabályok nélkül is alkalmazkodik a különböző képernyőméretekhez.

A terméklista és a konfigurátor komponensei rugalmas rácsszerkezetben jelennek meg, amely kisebb kijelzőkön automatikusan egymás alá rendezi az elemeket. A mobil nézet működése a Bootstrap beépített töréspontjain (breakpoint) alapul.

A projektben egyedi CSS használata minimális mértékben történt, elsősorban kisebb vizuális finomhangolások céljából.

4.4. WPF Desktop alkalmazás

Az adminisztrációs felület WPF technológiával készült, amely lehetővé teszi a törzsadatok karbantartását és a rendszer adminisztratív kezelését.

4.4.1. MVVM architektúra és Adatkötés

A WPF alkalmazás MVVM (Model-View-ViewModel) architektúrát követ.

Az adatkötés (Data Binding) segítségével a felhasználói felület automatikusan frissül a ViewModel állapotának változásakor. Ez tiszta és jól karbantartható kódszerkezetet eredményez.

4.4.2. Törzsadat-karbantartó modulok

A desktop alkalmazás lehetőséget biztosít:

- új testformák rögzítésére
- finish elemek feltöltésére
- pickguard elemek kezelésére
- nyak típusok módosítására
- termékek adminisztrálására

Ez a megoldás lehetővé teszi a webshop dinamikus bővítését anélkül, hogy a frontend vagy a backend forráskódját módosítani kellene.

5. fejezet

Minőségbiztosítás és Tesztelés

A szoftverfejlesztési folyamat során kiemelt szerepet kapott a rendszer megbízható működésének biztosítása. A minőségbiztosítás célja annak ellenőrzése volt, hogy az alkalmazás megfelel a meghatározott követelményeknek, stabilan működik különböző környezetekben, valamint felhasználói szempontból is megfelelő élményt nyújt.

A tesztelési folyamat több szinten történt: egységtesztek, API-tesztelés, felhasználói felület vizsgálata, valamint a fejlesztés során feltárt hibák dokumentálása és javítása.

5.1. Unit tesztek (Egységtesztelés)

Az egységtesztelés során a backend alkalmazás vezérlőinek (Controller-ek) működését ellenőriztük automatizált tesztek segítségével. A tesztek XUnit keretrendszerrel készültek, és egy memóriában futó adatbázist (InMemory Database) használtak, így a tesztelés független volt a valós adatbázistól.

A tesztek célja az volt, hogy ellenőrizzék az egyes API végpontok helyes működését különböző bemenetek esetén, beleértve a hibás és érvényes kéréseket is.

A tesztelt főbb funkciók:

- **Termékek lekérdezése:** Ellenőrzésre került, hogy a rendszer csak a nem üres képeket adja vissza.
- **Egy termék lekérdezése:** Megvizsgáltuk, hogy létező termék esetén helyes adatot ad vissza, nem létező esetén pedig megfelelő hibát (NotFound).
- **Regisztráció:** Teszteltük, hogy foglalt email cím esetén hibaüzenetet ad, új felhasználó esetén pedig sikeresen létrehozza a rekordot, megfelelő jelszó titkosítással.
- **Bejelentkezés:** Ellenőriztük a helyes és hibás belépési adatok kezelését, valamint a JWT token generálását.

- **Kijelentkezés:** A logout végpont helyes válaszát vizsgáltuk.
- **Kosár kezelés:** Teszteltük a kosárhoz adást és az összár helyes kiszámítását, valamint az autentikáció nélküli hozzáférés tiltását.
- **Kedvencek kezelése:** Ellenőriztük a termékek kedvencekhez adását és lekérdezését.
- **Egyedi gitár mentése:** Vizsgáltuk, hogy a konfiguráció mentése sikeresen létrehozza az új rekordot.

Az egységtesztek segítségével biztosítottuk, hogy a backend logika megbízhatóan működjön, és a kritikus funkciók hibamentesen viselkedjenek különböző körülmények között.

5.2. API tesztelés (Postman/Swagger)

A backend REST API végpontjainak ellenőrzése Postman és Swagger eszközök segítségével történt. A tesztelés célja annak vizsgálata volt, hogy a kliensalkalmazás és a szerver közötti kommunikáció megfelelően működik-e.

Az API tesztelés során az alábbi funkciók kerültek ellenőrzésre:

- felhasználói regisztráció és bejelentkezés végpontok,
- terméklista lekérdezése és szűrése,
- egyedi gitár konfiguráció mentése,
- kosárkezelési műveletek,
- rendelés létrehozása és lekérdezése.

A Swagger felület lehetőséget biztosított az API végpontok gyors áttekintésére és dokumentálására, míg a Postman segítségével különböző tesztesetek és hibás bemenetek is kipróbálhatók voltak.

5.3. Felhasználói felület és Reszponzivitás tesztelése

A frontend alkalmazás tesztelése során a hangsúly a felhasználói élményen és a különböző eszközökön történő megfelelő megjelenítésen volt.

A vizsgálat kiterjedt:

- a guitar builder vizuális működésére és az elemek helyes megjelenítésére,

– a szűrők és keresési funkciók működésére,

– a kosár és rendelési folyamat használhatóságára,

– reszponzív megjelenésre mobil, tablet és asztali nézetben.

A tesztelés több modern böngészőben történt (Google Chrome, Microsoft Edge, Mozilla Firefox), biztosítva a platformfüggetlen működést.

5.4. Hibajegyzék és javítások

A fejlesztés során felmerült hibák dokumentálása folyamatosan történt. A hibák azonosítása manuális tesztelés és felhasználói próbák során történt, majd prioritás szerint kerültek javításra.

A leggyakoribb problémák a következők voltak:

– hibás adatbázis kapcsolatok és idegen kulcs beállítások,

– nem megfelelő szűrési logika a frontend oldalon,

– eltérő adatstruktúrák a frontend és backend között,

– vizuális elcsúszások különböző képernyőméreteken.

A hibák javítása iteratív módon történt, amely során minden módosítás után újratesztelés biztosította, hogy a korábban működő funkciók ne sérüljenek. Ez a folyamat hozzájárult a rendszer stabil és megbízható működéséhez.

A tesztek konkrét megvalósítására példák a dokumentum végén, a mellékletek között találhatók (lásd 8.1, 8.2 és 8.3 ábrák).

6. fejezet

Felhasználói útmutató

6.1. Rendszerkövetelmények és Telepítés

A projekt három különálló alkalmazásból áll, amelyek külön GitHub repository-ban találhatók:

- Frontend – React alapú webalkalmazás
- Backend – REST API (Swagger dokumentációval)
- WPF alkalmazás – adminisztrációs felület

A rendszer megfelelő működéséhez mindhárom komponensnek futnia kell, valamint az adatbázis szervernek is elérhetőnek kell lennie.

Rendszerkövetelmények

A fejlesztői környezet futtatásához az alábbi szoftverek szükségesek:

- XAMPP (Apache + MySQL)
- Node.js és npm
- Visual Studio
- .NET fejlesztői környezet
- Modern webböngésző

Adatbázis indítása

Első lépésként el kell indítani a XAMPP vezérlőpultot, majd aktiválni kell a következő szolgáltatásokat:

- Apache
- MySQL

Az adatbázis importálása a phpMyAdmin felületen keresztül történik. Az adatbázis betöltése után a backend alkalmazás képes lesz csatlakozni az adatforráshoz.

Backend indítása

A backend projektet Visual Studio környezetben kell megnyitni.

Indítás előtt ajánlott a projekt újrafordítása (Rebuild), hogy minden függőség megfelelően betöltődjön.

A backend futtatása után a Swagger felület automatikusan megnyílik a böngészőben, ahol az API végpontok tesztelhetők.

A backend biztosítja az adatkapcsolatot az adatbázis és a kliensalkalmazások között.

Frontend indítása

A frontend egy React alapú webalkalmazás.

A projekt mappájában a szükséges csomagokat npm segítségével kell telepíteni:

```
npm install
```

A projekt működéséhez szükséges könyvtárak:

```
npm install react-router-dom
npm install bootstrap
npm install bootstrap-icons
npm install html2canvas
```

A frontend fejlesztői szerver a következő paranccsal indítható:

```
npm start
```

A webalkalmazás alapértelmezés szerint a következő címen érhető el:

```
http://localhost:3000
```

A frontend induláskor automatikusan lekéri az adatokat a backend API-ból.

WPF alkalmazás indítása

Az adminisztrációs felület egy külön WPF alkalmazás formájában valósult meg, amely szintén külön GitHub repository-ban található.

A projektet Visual Studio-ban kell megnyitni, majd futtatni. Az alkalmazás a backend API segítségével kommunikál az adatbázissal.

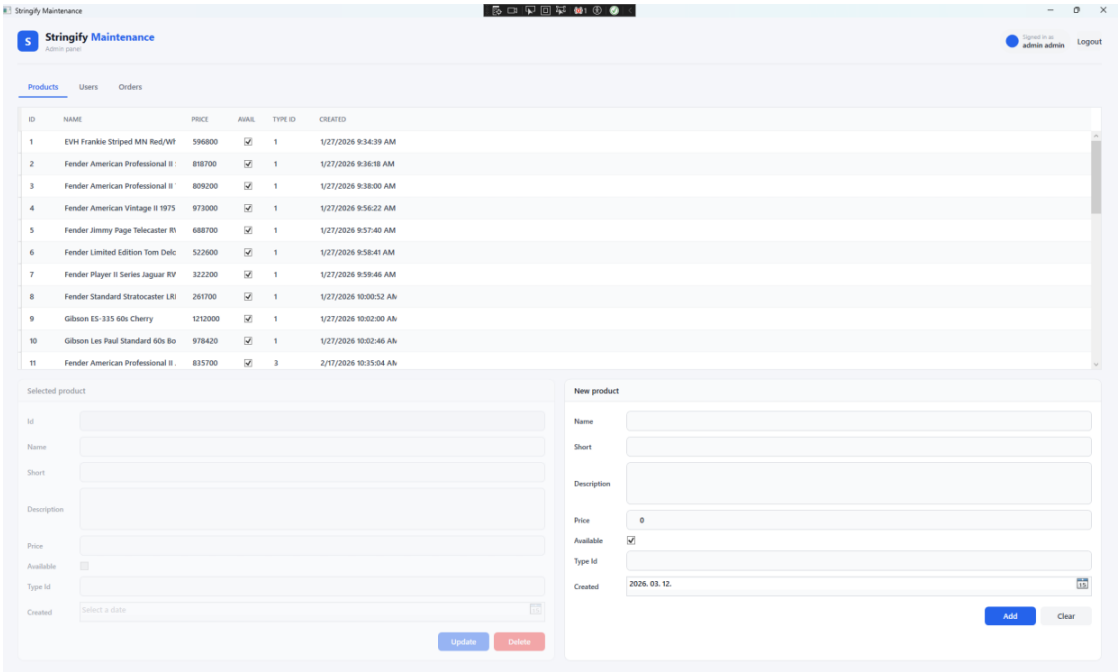
Fontos, hogy a WPF alkalmazás használata előtt a következő szolgáltatások már fussanak:

- XAMPP (MySQL)
- Backend API

6.2. A szoftver használata (Web/WPF)

A rendszer két külön felhasználói felületet biztosít:

- Webes felület (frontend)
- Adminisztrációs felület (WPF)



6.1. ábra. WPF Admin felület

WPF admin felület

A WPF alkalmazás adminisztrációs célokat szolgál, például termékek és adatok kezelésére.

Az admin felület használatához megfelelő jogosultság szükséges. A felhasználónak admin jogosultsággal kell rendelkeznie, amely az adatbázisban egy magasabb jogosultsági szinttel van jelölve.

Amennyiben egy felhasználó adminisztrátori hozzáférést kap, a jogosultsági értéket az adatbázisban módosítani kell (például 1 helyett 9 értékre).

Ezzel a jogosultsággal a felhasználó be tud jelentkezni a WPF admin felületre, és elérheti az adminisztrációs funkciókat.

7. fejezet

Összegzés

7.1. Projekt értékelése

A projekt célja egy modern, webalapú gitárwebshop és egyedi gitártervező rendszer létrehozása volt, amely lehetővé teszi a felhasználók számára hangszerek böngészését, szűrését, rendelését, valamint saját konfigurációjú gitár összeállítását.

A fejlesztés során sikerült egy teljes értékű, több rétegből álló alkalmazást megvalósítani:

- **Frontend réteg**: React alapú felhasználói felület, amely reszponzív kialakítással biztosítja a használhatóságot különböző eszközökön.
- **Backend API**: REST alapú szolgáltatás Swagger dokumentációval, amely kezeli a termékeket, rendeléseket, felhasználókat és az egyedi gitár konfigurációkat.
- **Adatbázis**: relációs adatmodell, amely támogatja a webshop működését, a jogosultságkezelést, valamint a gitárépítő modul adatszerkezetét.

A rendszer egyik legfontosabb eleme az egyedi gitártervező funkció, amelyben a felhasználó különböző komponensekből (testforma, nyak, pickguard, felületkezelés stb.) állíthatja össze hangszerét. A moduláris felépítés lehetővé teszi az alkatrészek későbbi bővítését és új konfigurációk hozzáadását.

A projekt során külön figyelmet kapott:

- a felhasználói élmény (UX),
- a szűrési és rendezési lehetőségek kialakítása,
- az adminisztrációs és adatkezelési logika,
- valamint a skálázható adatbázis-struktúra.

A fejlesztés során több technikai kihívás is felmerült — például az egyedi konfigurátor frontend logikájának megtervezése, a dinamikus szűrők működése, valamint az adatbázis megfelelő normalizálása — azonban ezek sikeresen megoldásra kerültek iteratív fejlesztéssel és teszteléssel.

Összességében a projekt elérte a kitűzött célokat: létrejött egy működőképes webshop rendszer és egy alapjaiban bővíthető gitárépítő platform.

7.2. Továbbfejlesztési lehetőségek

A rendszer jelenlegi állapotában stabil alapot biztosít további funkciók bevezetéséhez. A jövőbeni fejlesztések több irányba is kiterjeszthetők.

Gitártervező bővítése

- Valós idejű 3D megjelenítés (WebGL vagy Three.js használatával).
- Több gitártípus támogatása (pl. jazzmaster, basszus modellek).
- Alkatrészek kompatibilitásának automatikus ellenőrzése.
- Egyedi konfigurációk megosztása vagy publikálása.

Webshop funkciók fejlesztése

- Online fizetési rendszerek integrálása.
- Rendeléskövetési rendszer.
- Felhasználói értékelések
- Kupon- és akciókezelés admin felületen.

Felhasználói élmény és teljesítmény

- Képek és API hívások optimalizálása.
- Cache-elési megoldások bevezetése.
- Progresszív webalkalmazás (PWA) támogatás.

Adminisztráció és analitika

- Statisztikai dashboard az eladások elemzésére.
- Felhasználói viselkedés elemzése.
- Jogosultsági rendszer további finomítása.

A projekt jelenlegi architektúrája lehetővé teszi ezen fejlesztések fokozatos implementálását anélkül, hogy az alap rendszer működését jelentősen módosítani kellene. Ennek köszönhetően a rendszer hosszú távon is fenntartható és továbbfejleszthető.

Köszönetnyilvánítás

Szeretnénk köszönetet mondani mindazoknak az oktatóknak, akik szakmai tudásukkal és támogatásukkal hozzájárultak a vizsgaremekem elkészítéséhez.

Külön köszönettel tartozunk az IKT projektmunka, illetve Backend programozás tantárgy oktatójának, *[Kerényi Róbert Nándor]* tanár úrnak a projekt során nyújtott iránymutatásért és segítségért.

Köszönet illeti a frontend fejlesztést oktató *[Négyesi Gábor]* tanár urat a React alapú webes felület kialakításához nyújtott segítségéért.

Végül szeretnénk megköszönni az adatbázis tantárgy oktatójának, *[Kasza László Róbert]* tanár úrnak az adatbázis tervezésével és megvalósításával kapcsolatos segítséget.

8. fejezet

Mellékletek

8.1. Backend unit tesztek

```
[Fact]
1 reference
public async Task ProductsController_GetAll_FiltersEmptyImages()
{
    var db = TestHelpers.CreateInMemoryContext(nameof(ProductsController_GetAll_FiltersEmptyImages));

    var product = new Termek
    {
        Id = 1,
        Nev = "Guitar",
        Leiras = "Long",
        RovidLeiras = "Short",
        Ar = 100,
        Elerheto = true,
        Letrehozva = DateTime.UtcNow
    };

    var images = new TermekKepek
    {
        TermekId = 1,
        Kep1 = "img1",
        Kep2 = "",
        Kep3 = "img3",
        Kep4 = " ",
        Kep5 = ""
    };

    product.TermekKepek = images;
    images.Termek = product;

    db.Termekek.Add(product);
    await db.SaveChangesAsync();

    var controller = new Stringify.Api.Controllers.ProductsController(db);
    var actionResult = await controller.GetAll();
    var ok = Assert.IsType<OkObjectResult>(actionResult.Result);
    var list = Assert.IsAssignableFrom<List<ProductDto>>(ok.Value);

    Assert.Single(list);
    Assert.Equal(2, list[0].Images.Count);
    Assert.Contains("img1", list[0].Images);
    Assert.Contains("img3", list[0].Images);
}
```

8.1. ábra. Termék lekérdezés teszt (képszűrés)

```
[Fact]
1 reference
public async Task LoginController_Login_ReturnsOkOrBadRequest_WhenCredentialsProvided()
{
    var db = TestHelpers.CreateInMemoryContext(nameof(LoginController_Login_ReturnsOkOrBadRequest_WhenCredentialsProvided));
    var salt = "test-salt";
    var raw = "password";
    var tmpHash = Program.CreateSHA256(raw + salt);
    var finalHash = Program.CreateSHA256(tmpHash);

    db.Users.Add(new User
    {
        Id = 1,
        Email = "login@example.com",
        Nev = "Login",
        Jelszo = finalHash,
        Salt = salt,
        Aktiv = 1,
        Jogosultsag = 1
    });
    await db.SaveChangesAsync();

    var config = TestHelpers.CreateConfiguration(new Dictionary<string, string?>
    {
        ["Jwt:Key"] = "default-key-1234567890",
        ["Jwt:Issuer"] = "test",
        ["Jwt:Audience"] = "test",
        ["Jwt:ExpirationMinutes"] = "60"
    });

    var controller = new LoginController(db, config);
    var result = await controller.Login(new LoginDTO { Email = "login@example.com", TmpHash = tmpHash });

    if (result is OkObjectResult ok)
    {
        var loggedUser = Assert.IsType<LoggedUser>(ok.Value);
        Assert.Equal("Login", loggedUser.Name);
        Assert.Equal("login@example.com", loggedUser.Email);
        Assert.False(string.IsNullOrWhiteSpace(loggedUser.Token));
    }
    else if (result is BadRequestObjectResult bad)
    {
        var message = bad.Value?.ToString() ?? string.Empty;
        Assert.Contains("Hiba", message, StringComparison.OrdinalIgnoreCase);
    }
    else
    {
        throw new InvalidOperationException("Expected OkObjectResult or BadRequestObjectResult from Login");
    }
}
```

8.2. ábra. Bejelentkezési folyamat tesztelése

```
[Fact]
1 reference
public async Task ProductsController_GetAll_FiltersEmptyImages()
{
    var db = TestHelpers.CreateInMemoryContext(nameof(ProductsController_GetAll_FiltersEmptyImages));

    var product = new Termek
    {
        Id = 1,
        Nev = "Guitar",
        Leiras = "Long",
        RovidLeiras = "Short",
        Ar = 100,
        Elerheto = true,
        Letrehozva = DateTime.UtcNow
    };

    var images = new TermekKepek
    {
        TermekId = 1,
        Kep1 = "img1",
        Kep2 = "",
        Kep3 = "img3",
        Kep4 = " ",
        Kep5 = ""
    };

    product.TermekKepek = images;
    images.Termek = product;

    db.Termekek.Add(product);
    await db.SaveChangesAsync();

    var controller = new Stringify.Api.Controllers.ProductsController(db);
    var actionResult = await controller.GetAll();
    var ok = Assert.IsType<OkObjectResult>(actionResult.Result);
    var list = Assert.IsAssignableFrom<List<ProductDto>>(ok.Value);

    Assert.Single(list);
    Assert.Equal(2, list[0].Images.Count);
    Assert.Contains("img1", list[0].Images);
    Assert.Contains("img3", list[0].Images);
}
```

8.3. ábra. Kosár működésének tesztelése